INFORMATION GUIDE

CareBridge Hospital System

Project structure, folder guide, and backend walkthrough

A practical map of the PHP application, role-based front end, MySQL schema, and the path an API request takes through the system.

Project type: academic hospital patient-processing prototype | Reviewed: November 2026

Important: This is an academic prototype, not a production or clinical system. Use fictional demonstration data only. Payment approvals are simulated, and the presence of a module does not mean every workflow has been fully end-to-end tested.

What the project does

CareBridge coordinates administrative and clinical workflow screens around staff accounts, patients, visits, appointments, triage, consultations, diagnostic orders and results, referrals, billing, prescriptions, dispensing, and audit history. Staff sign in and work through pages appropriate to their role.

At a glance

Folder / area	Purpose
Browser UI	PHP-rendered pages under frontend/ and auth/, with CSS and JavaScript in frontend/assets/.
API	A single PHP entry point at backend/index.php dispatches JSON requests to route handlers.
Business logic	Controllers authorize actions, validate input, run prepared SQL statements, and return JSON.
Persistence	MySQL/InnoDB tables are declared in database/hosp.sql; additive changes live in database/migrations/.

How the backend works

The application uses a browser-to-PHP-to-MySQL flow. PHP pages render the workspace shell; JavaScript calls the API for data and state changes. The API is centralized rather than split into separate server processes.

Typical request lifecycle

1. Render a page Apache runs a PHP page such as frontend/reception/index.php. Shared layout partials provide navigation and page structure; a page middleware checks that a signed-in user is allowed to see it.

2. Start a browser request Page JavaScript uses fetch() to call backend/index.php with a route query parameter, for example ?route=/api/reception/dashboard. Same-origin session cookies identify the signed-in user; state-changing requests also include the CSRF token.

3. Initialize the API backend/index.php loads database configuration, response helpers, authentication middleware, controllers, and the API router. It starts the secure PHP session, normalizes the route, and dispatches the HTTP method and path.

4. Match and authorize backend/routes/api.php defines supported routes and methods, including parameterized routes. Handler/controller code checks the active session, role permissions, input shape, and CSRF token where needed.

5. Read or change data A controller obtains a PDO MySQL connection and uses prepared statements. Multi-step operations use transactions where consistency matters; relational constraints connect users, patients, visits, orders, payments, and other records.

6. Return JSON and update UI The response helpers send a consistent success/error envelope and no-store headers. Browser JavaScript reads the JSON, shows feedback, and updates tables, queues, forms, and status indicators.

Example route

```
GET /backend/index.php?route=%2Fapi%2Freception%2Fdashboard
```

The reception page JavaScript requests its dashboard data. The API entry point maps the route to a reception handler; the handler checks receptionist/administrator access, queries the database, and returns JSON for the page to render.

Session and account safeguards

Authentication uses PHP sessions with cookie protections, password_hash()/password_verify(), and CSRF tokens for writes. Role checks happen on the server, not just by hiding links in the browser. Staff accounts created by administrators receive a temporary password and must change it at first sign-in. Administrators can issue a generated temporary password reset; account events are recorded in the existing audit log.

The frontend improves usability, but server authorization and validation remain the security boundary.

Folder-by-folder guide

Repository root and backend

Folder / area	Purpose
index.php	Project landing page and entry link into the application.
README.md	XAMPP setup instructions, database configuration, administrator bootstrap, API notes, and project limitations.
.gitignore	Git ignore rules for local-only files.
test_e2e_http.php	HTTP-oriented end-to-end test script for application flows.
test_radiology_flow.php	Focused test script for radiology workflow behavior.
backend/index.php	API bootstrap and single entry point; loads dependencies, starts the session, normalizes route input, then invokes dispatchApiRequest().
backend/config/	app.php contains app/base URL and registration settings. database.php reads environment variables or an ignored database.local.php and creates PDO connections.
backend/controllers/	Feature handlers and SQL/business rules: auth, admin, reception, nursing, doctor, emergency, testing, radiology, cardiac, specialists, therapy, billing, pharmacy, and monitoring/audit.
backend/helpers/	Shared response formatting and billing policy helpers reused by controllers.
backend/middleware/	Session and CSRF logic, authenticated-user lookups, administrator checks, and page-access guards for role workspaces.
backend/routes/	api.php declares supported API paths/methods, validates route/method matches, and dispatches to handlers.
backend/setup/	CLI-only create_admin.php bootstraps the first administrator with a hashed password and refuses repeat initial setup.
backend/models/ and backend/services/	Currently placeholders (.gitkeep only). Most data access and workflow logic currently lives in feature controllers rather than dedicated model/service classes.

Database and authentication folders

Folder / area	Purpose
database/hosp.sql	Creates the MySQL database schema and role catalogue. User accounts are intentionally not seeded with demo credentials.
database/migrations/	Incremental SQL updates for existing databases; includes module changes and the staff-account must_change_password field. Apply required migrations once after importing the baseline schema.
auth/login.php	Staff sign-in screen; frontend/assets/js/auth.js calls the login/session APIs.
auth/signup.php	Legacy registration entry redirects to administrator-managed staff account creation; public registration remains disabled.

Front end and supporting folders

The frontend/ directory holds role-oriented PHP screens. Most feature folders contain page files plus underscore-prefixed layout/partial files that are included by pages to reuse navigation, headers, and view fragments.

Folder / area	Purpose
frontend/admin/	Administrator overview, staff-account administration, fee management, reports, and audit-log viewing.
frontend/reception/	Patient search/registration, records, appointments, arrivals, visit queues, referrals, and the reception workspace.
frontend/nursing/	Nursing queues, assessments, completed work, referrals, and patient-care screens.
frontend/doctor/	Doctor dashboard, visit queue, consultations, clinical records, history, diagnoses, and referrals.
frontend/emergency/	Emergency department pages, with distinct doctor and nurse workspaces, assessments, handovers, treatment, orders, outcomes, and history.
frontend/testing-desk/	Test-order queue, routing to departments, in-progress/completed work, and order history.
frontend/laboratory/	Laboratory orders, specimens, results, processing, and history.
frontend/radiology/	Imaging workflow and radiology image access.
frontend/cardiac/	Cardiac orders, imaging, examination processing, results, reports, and history.
frontend/specialists/	Specialist referral work and outside-referral views.
frontend/therapy/	Therapy referrals, assessments, treatment plans, sessions, progress, patients, and reports.
frontend/billing/	Ticket payments, payment queues, fee-related screens, receipts, and recent payments.
frontend/pharmacy/	Prescriptions, dispensing, medicine inventory, stock activity, payments, receipts, alerts, and reports.
frontend/assets/css/	Stylesheets for shared authentication/staff layouts and individual admin/department modules.
frontend/assets/js/	Browser code for API requests and interactions: auth.js, admin.js, and per-module scripts for queues, forms, and views.
frontend/change-password.php	Temporary-password first-sign-in screen; JavaScript submits the authenticated change-password API request.

Other root-level assets

Folder / area	Purpose
assets/	Shared top-level css/, images/, and js/ directories are currently placeholders; most active browser assets are under frontend/assets/.
CareBridge Hospital System Presentation.pptx	Project presentation deck.
PresentationVideo.mp4 / PresentationVideo2.mp4	Project presentation/demo media.
patient hospital system report.pdf	Existing assignment/report document; separate from this information guide.
information guide.pdf	This generated guide to the repository and backend flow.

How the pieces work together

A staff member signs in, then the role-specific pages use the same session-backed API. The user interface organizes work by department, while the backend validates each request and persists the resulting state in shared relational tables.

Typical patient-workflow thread

Reception registers or finds a patient and opens a visit. Nursing can assess and route the patient; a doctor can document a consultation, diagnosis, test order, prescription, or referral. Testing-desk and diagnostic departments process orders and return results. Billing records fees/payments; pharmacy handles prescriptions and dispensing. Audit records capture selected staff actions across workflows. Exact paths vary by module and the repository notes that not every workflow has been end-to-end verified.

Database concepts

The schema is centered on roles and users for staff identity; patients and visits for care episodes; appointments and department-specific assessment/order/result/referral tables for workflow; bills, bill items, payment, and fee catalogue tables for charging; prescriptions, dispensings, and stock movement tables for pharmacy; and audit_log for accountability. Foreign keys and indexes connect records and support common lookups.

Local setup outline

1. Put the project under the XAMPP htdocs directory and start Apache/MySQL.
2. Import database/hosp.sql; for an existing database, apply the required SQL files in database/migrations/ once.
3. Configure HOSPITAL_DB_NAME, HOSPITAL_DB_USER, HOSPITAL_DB_PASSWORD (and optional host/port), or use backend/config/database.local.php. Keep credentials out of Git.
4. Run backend/setup/create_admin.php from the CLI to create the first administrator.
5. Open the project root or auth/login.php in a browser. Administrators create staff accounts.

No URL rewriting is required: the browser sends a route query parameter to backend/index.php. Project URLs are designed to work from a subfolder under htdocs.

Important boundaries

This codebase is an academic prototype. Some payment/medical-aid outcomes are simulated; demo fee and medicine values are not real; complete authenticated workflows have not been verified. Do not use it to make clinical decisions or store real patient information without a proper production-grade review, hardening, operational controls, and privacy/compliance assessment.

Quick mental model

PHP pages render department workspaces; **JavaScript** requests and presents JSON; **backend/index.php + routes** select the handler; **middleware** authenticates/authorizes; **controllers** validate and run SQL; **MySQL** stores shared records; **audit_log** provides a review trail.